

Technical Writeup

Technical Writeup: Vulnerable Browser Extension

CS 4277 — Final Project Team Members: Maret Rudin-Aulenbach, Jimmy Baek, Juan Guerrero

1. Project Overview

This project is a Chrome browser extension, built on Manifest V3, that summarizes webpage content for the user. The extension reads text from the currently active tab, generates a short summary, and stores a history of past summaries in `chrome.storage.local`. The user can review their history through the extension popup or a dedicated options page.

The purpose of the project is educational: we intentionally introduced two distinct security vulnerabilities into a “vulnerable” build of the extension, then developed a hardened “secure” build that addresses each vulnerability with targeted defenses. A set of demo pages (`malicious_demo/`) lets a presenter simulate both attacks live, making the risks tangible for an audience.

Extension Architecture

The extension consists of three main components:

Component	File	Role
Popup UI	<code>popup.html / popup.js</code>	User-facing toolbar popup with “Summarize” and “View History” buttons.
Background Service Worker	<code>background.js</code>	Receives text from the content script or popup, produces a summary, and manages persistent history storage via <code>chrome.storage.local</code> .
Content Script	<code>content.js</code>	Injected into every page (<code><all_urls></code>). Handles

Component	File	Role
Options Page	options.html / options.js	page-level text collection and cross-context messaging.
		Displays stored summary history inside the extension's own UI context.

The two builds — `vulnerable/` and `secure/` — share this architecture but differ in how they collect data and who they communicate with.

2. Attack 1: Message-Based History Exfiltration

2.1 Vulnerability Root Cause

The vulnerable extension's content script (`vulnerable/content.js`) registers a `window.addEventListener("message", ...)` handler that listens for incoming `postMessage` events. Critically, this handler **performs no origin validation** — it processes messages from any source, including arbitrary web pages the user visits:

```
// vulnerable/content.js
window.addEventListener("message", async (event) => {
  // ❌ No origin check – event.origin is never inspected.
  if (event.data?.type === "GET_HISTORY") {
    chrome.runtime.sendMessage({ type: "GET_HISTORY_FOR_POPUP" },
      (response) => {
        window.postMessage(
          { type: "HISTORY_RESPONSE", history: response?.history || [] },
          "*" // ❌ Replies to "*" – any listener on the page receives the data.
        );
      });
  }
});
```

The `window.postMessage / message` API is designed for cross-origin communication between frames and windows. Because the content script runs in the same page context as the website's own JavaScript, a malicious page can simply call `window.postMessage({ type: "GET_HISTORY" }, "*")` and wait for the content script to reply with the user's entire summary history. The background service worker (`vulnerable/background.js`) also lacks any sender validation — it responds to `GET_HISTORY_FOR_POPUP` from any caller without checking whether the request originated from the extension popup or an untrusted content script.

Furthermore, the response is posted back via `window.postMessage(..., "*")`, broadcasting the history to every listener on the page, including the attacker's script.

2.2 Attack Scenario

1. The user browses the web normally and uses the vulnerable extension to summarize several pages. Summaries accumulate in `chrome.storage.local`.
2. The user visits a malicious website (demonstrated by `malicious_demo/index.html`, disguised as a news article).
3. The malicious page runs a single line of JavaScript:

```
window.postMessage({ type: "GET_HISTORY" }, "*");
```

4. The content script, already injected on the page, receives this message, forwards it to the background worker, and posts the entire history back to the page.
5. The attacker's script captures the response and could silently `fetch()` it to an external server.

2.3 Defense Design Decisions

The secure extension eliminates this attack surface through three layered changes:

a) Remove the `postMessage` bridge entirely. The secure content script (`secure/content.js`) still registers a `window.addEventListener("message", ...)` handler, but it is a defensive no-op that filters by origin and processes nothing:

```
window.addEventListener("message", (event) => {  
  if (event.origin !== window.location.origin) return;  
  if (!event.data || typeof event.data !== "object") return;  
  // Intentionally does NOT expose history or summary APIs.  
});
```

No page-level script can trigger a history lookup or summary via `postMessage` because the handler never forwards those requests.

b) Validate the sender in the background worker. The secure background script (`secure/background.js`) checks `sender.tab` when processing `GET_HISTORY_FOR_POPUP` messages:

```
if (message?.type === "GET_HISTORY_FOR_POPUP" && !sender.tab) {  
  // Only respond when the sender is NOT a content script (i.e.,  
  // it's the popup or options page).  
}
```

In Chrome's messaging model, requests from content scripts carry a `sender.tab` object, while requests from the popup or options page do not. By requiring `sender.tab`, the background worker ensures that only the extension's own privileged UI can retrieve history.

c) Restrict history viewing to extension-owned UI. History is only viewable by clicking the extension icon and selecting "View History" inside the popup, or by navigating to the extension's options page (`options.html`). Both run in the extension's own origin (`chrome-extension://...`), which web pages cannot access or impersonate.

3. Attack 2: Over-Collection of Sensitive DOM Data

3.1 Vulnerability Root Cause

The vulnerable extension’s content script and popup both use the function `collectEntireDomText()` to extract text for summarization. This function performs two excessively broad data collection operations:

```
// vulnerable/content.js
function collectEntireDomText() {
  const visible = document.body?.innerText || "";           // ✗
  // Grabs ALL visible text.
  const hiddenValues = Array.from(
    document.querySelectorAll("input, textarea")           // ✗
  // Scrapes ALL input values.
  ).map((el) => `${el.name || el.id || "field"}=${el.value || ""}`);
  // return [visible, hiddenValues.join("\n")].join("\n").trim();
}
```

`document.body.innerText` captures all rendered text on the page, which on a banking site would include account numbers, routing numbers, balances, and SSNs — even those that appear visually masked with CSS (e.g., `filter: blur(4px)`), because `innerText` reads the computed text content rather than its visual appearance.

`document.querySelectorAll("input, textarea")` enumerates every form element on the page. This includes: - `<input type="password">` fields containing plaintext passwords (the `.value` property always returns the cleartext regardless of the input type). - `<input type="hidden">` fields containing CSRF tokens, session IDs, internal API keys, and other secrets that are never displayed to the user.

Because the summarization pipeline passes this collected text directly to the background worker, which stores it in history, all of this sensitive data ends up persisted in extension storage. Combined with Attack 1, an attacker on a separate site could then exfiltrate this stored sensitive data remotely.

3.2 Attack Scenario

The demo page `malicious_demo/attack2.html` is a mock bank portal (“SecureBank Online”) that contains sensitive data in multiple DOM locations:

Data	DOM Location	Visual Appearance
Account numbers (4821-0039-7713)	Visible <code><div></code> text	Visible on screen
Routing number (091000019)	Visible <code><div></code> text	Visible on screen
	Visible <code><div></code> text	Visible on screen

Data	DOM Location	Visual Appearance
Account balances (\$24,817.43)		
SSN (123-45-6789)	<code><input type="hidden"></code>	Not rendered at all
User password (MyS3cretBanking!)	<code><input type="password"></code>	Masked with dots
Security answer (Fitzgerald)	<code><input type="password"></code>	Masked with dots
CSRF token	<code><input type="hidden"></code>	Not rendered
Session ID	<code><input type="hidden"></code>	Not rendered
Account token	<code><input type="hidden"></code>	Not rendered
Email address	Visible text	Visible on screen

When the vulnerable extension’s “Summarize Entire Page” button is clicked while viewing this page, `collectEntireDomText()` captures all of this data — including the password field values, hidden tokens, and SSN — and stores it in the summary history. The demo page’s “Run Attack 2” button simulates this same collection to visually demonstrate what would be captured, with sensitive values highlighted in red in a terminal-style overlay.

3.3 Defense Design Decisions

The secure extension addresses this vulnerability through two complementary defense layers:

a) Selection-only input (primary defense). Instead of scraping the entire DOM, the secure extension only summarizes text that the user has explicitly highlighted. The content script (`secure/content.js`) uses `window.getSelection()` instead of `document.body.innerText`:

```
function getUserSelectedText() {
  const live = window.getSelection()?.toString().trim() || "";
  return live || cachedSelection;
}
```

The popup button is labeled “Summarize Selection” (not “Summarize Entire Page”), and the popup UI includes a warning banner:

Highlight only non-sensitive text before summarizing. Do not select passwords, IDs, account numbers, or tokens.

If no text is selected, the extension refuses to proceed and displays an error message. This design shifts the control to the user: they choose exactly what goes into the summary, eliminating the automatic over-collection of passwords and hidden fields.

A selection-caching mechanism was also implemented to deal with a Chrome-specific UX challenge: when the user clicks the extension popup, Chrome shifts focus away from the page, which can clear the active selection. The content script listens for `mouseup` and `selectionchange` events to continuously cache the most recent non-empty selection, ensuring it persists through the focus shift.

b) Regex-based sensitive data filtering (defense-in-depth). Even with selection-only input, a user might accidentally select text that contains sensitive values, or a page might display sensitive data inline where it could be selected alongside safe text. As a last-resort layer, the secure background worker (`secure/background.js`) runs a `sanitizeSensitiveData()` function on all incoming text before generating a summary:

```
const SENSITIVE_PATTERNS = [
  { re: /eyJ[A-Za-z0-9_-]+\.\eyJ[A-Za-z0-9_-]+\.[A-Za-z0-9_-]+/g,
    label: "[JWT_REDACTED]" },
  { re: /\b\d{3}-\d{2}-\d{4}\b/
    g,
    label: "[SSN_REDACTED]" },
  { re: /\b(?:\d{4}[\s-]){3}\d{4}\b/
    g,
    label: "[CARD_REDACTED]" },
  { re: /\b0[2-9]\d{7}\b/
    g,
    label: "[ROUTING_REDACTED]" },
  { re: /(?:csrf|session|token|auth|api[_]?key|...)[=:\s]+.../
    gi,
    label: "[TOKEN_REDACTED]" },
  { re: /(?:password|passwd|pwd)[=:\s]+\S+/
    gi,
    label: "[PASSWORD_REDACTED]" },
  { re: /\b[0-9a-f]{32,}\b/
    gi,
    label: "[HEX_SECRET_REDACTED]" },
  { re: /[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/g,
    label: "[EMAIL_REDACTED]" }
];
```

This sanitizer catches common sensitive data patterns — SSNs, credit card numbers, JWT tokens, key=value pairs for passwords/CSRF/session tokens, lengthy hex secrets, routing numbers, and email addresses — and replaces them with descriptive `[*_REDACTED]` labels before the summary is stored. This ensures that even if sensitive data reaches the background worker, it is never persisted in cleartext.

4. Summary of Secure Design Principles

Principle	How It Is Applied
Least Privilege	The secure extension only reads user-selected text, not the entire DOM. No hidden inputs or password fields are ever accessed.
Strict Origin / Sender Validation	The content script rejects cross-origin <code>postMessage</code> events. The background worker verifies that history requests come from extension UI only (<code>!sender.tab</code>).

Principle	How It Is Applied
Explicit User Action	Summarization requires a deliberate text selection. The extension refuses to operate without one.
Defense in Depth	Even if a user selects sensitive text, the regex sanitizer in the background worker redacts common secret patterns before storage.
UX Safety Cues	The popup displays a warning banner reminding users not to select sensitive information. The button label (“Summarize Selection”) communicates the scope of the action.
Minimal Attack Surface	The secure content script does not expose any functionality via <code>window.postMessage</code> . History is only accessible from the extension’s own options page.

5. Project Structure

```

cs4277finalproject/
├── vulnerable/                                # Intentionally insecure extension
│   ├── manifest.json
│   └── background.js                          # No sender validation, processes any
message
│   ├── content.js                            # Over-collects DOM text, open
postMessage bridge
│   ├── popup.html / popup.js
│   └── options.html / options.js
├── secure/                                    # Defended extension
│   ├── manifest.json
│   └── background.js                          # Sender-validated, regex sanitizer
for sensitive data
│   ├── content.js                            # Selection-only, cached selection,
closed postMessage
│   ├── popup.html / popup.js                # Selection-aware UI with warning
banner
│   └── options.html / options.js
├── malicious_demo/
│   ├── index.html                            # Attack 1 demo – fake news site that
steals history
│   └── attack2.html                          # Attack 2 demo – mock bank portal
with sensitive DOM data
├── README.md
└── writeup.md                                # This document

```

6. How to Run the Demos

Prerequisites

- Google Chrome (version 110 or later recommended)
- Developer Mode enabled at `chrome://extensions`

Loading an Extension

1. Navigate to `chrome://extensions`.
2. Toggle **Developer Mode** on.
3. Click **Load unpacked** and select either the `vulnerable/` or `secure/` directory.
4. Use one extension at a time to avoid confusion.

Demonstrating Attack 1

1. Load the **vulnerable** extension.
2. Visit several pages and click “Summarize Entire Page” to build up some history.
3. Open `malicious_demo/index.html` in Chrome.
4. Click the “**Run Attack 1: Steal Extension History**” button.
5. Observe the stolen history displayed in a red-bordered panel.

Demonstrating Attack 2

1. Load the **vulnerable** extension.
2. Open `malicious_demo/attack2.html` in Chrome (the mock bank portal).
3. Click “**Run Attack 2: Over-Collect Page Content**” on the demo page.
4. Observe the terminal overlay showing all captured data — passwords, tokens, SSNs are highlighted in red.
5. Alternatively, click “Summarize Entire Page” in the extension popup and then “View History” to see the sensitive data stored in the extension’s history.

Demonstrating the Defenses

1. Unload the vulnerable extension and load the **secure** extension.
2. Attempt Attack 1: open `malicious_demo/index.html` and click the attack button — no response is received.
3. Attempt Attack 2: navigate to `malicious_demo/attack2.html`, click “Summarize Selection” without selecting text — the extension refuses. Select only safe text (like a transaction description), summarize, and verify no passwords or tokens appear in the summary or history.

7. Team Contributions

Team Member	Contributions
Juan Guerrero	Built the foundational extension architecture and boilerplate: Manifest V3

Team Member	Contributions
Maret Rudin-Aulenbach	configuration, popup UI, background service worker, content script scaffolding, options page, and the storage layer. Developed the vulnerable version of the extension that serves as the baseline for both attacks.
	Designed and implemented Attack 1 (message-based history exfiltration) including the vulnerable <code>postMessage</code> bridge in the content script. Created the corresponding demo page (<code>malicious_demo/index.html</code>). Developed the Attack 1 defense : removing the open <code>postMessage</code> bridge, adding origin validation, restricting history access to authorized extension UI only (popup and options page), and implementing sender validation (<code>!sender.tab</code>) in the background worker.
	Designed and implemented Attack 2 (over-collection of sensitive DOM data) including the <code>collectEntireDomText()</code> function that scrapes all visible text and hidden input values. Created the Attack 2 demo page (<code>malicious_demo/attack2.html</code>) — a mock bank portal with embedded sensitive data.
Jimmy Baek	Developed the Attack 2 defense : replacing full-DOM collection with selection-only input (using <code>window.getSelection()</code> with a caching mechanism), adding the regex-based sensitive data sanitizer (<code>sanitizeSensitiveData()</code>) in the background worker as a defense-in-depth layer, and implementing the UX elements (warning banner, selection-required flow) in the secure popup.

8. Conclusion

This project demonstrates that browser extensions occupy a uniquely privileged position in the browser: they can read page content that is invisible to users (hidden inputs, password fields, CSS-masked elements) and persist it across sessions. When combined with insecure messaging patterns, this data can be exfiltrated to any website the user visits.

The defenses applied follow established secure-design principles — least privilege, explicit user action, origin validation, and defense in depth — to systematically eliminate each attack vector. The combination of user-controlled input (selection-only summarization), strict message validation (sender/origin checks), access control (history restricted to extension UI), and automated sanitization (regex filtering) provides multiple independent barriers, so that a failure in any single layer does not compromise user data.